vaultctl

User Manual · vaultctl v1.5.0

cdds-ab

April 2026

cdds-ab · github.com/cdds-ab/vaultctl

Contents

1 Introduction

vaultctl is a command-line interface for managing Ansible Vault secrets with structured metadata. It wraps the standard `ansible-vault` tool in a workflow built around three ideas:

1. **Secrets and metadata are separate.** The encrypted file holds values; an unencrypted sibling file holds descriptions, rotation schedules, expiry dates, and consumers. The metadata is reviewable and version-controllable on its own.
2. **Schemas are first-class.** Optional CUE schemas describe the shape of valid vault content. Validation and drift detection turn structural changes into deliberate, auditable steps.
3. **Local by default.** All operations run on the developer's machine. There is no cloud backend, no analytics, no telemetry.

This manual is a guided tour of the tool. The README on GitHub gives the quick reference; this document is meant to be read end to end.

1.1 When to Use vaultctl

vaultctl earns its keep on projects where:

- Ansible Vault is already the encryption-at-rest format and changing that isn't on the table.
- More than a handful of secrets exist, and tracking why each one matters is a real concern.
- Rotation or expiry timelines need to be visible to humans and to CI alike.
- A schema-driven gate on what counts as valid vault content is desirable.

It is **not** a replacement for HashiCorp Vault, AWS Secrets Manager, or any online secret broker. vaultctl is a local CLI on top of files.

1.2 What This Manual Covers

The chapters that follow walk through installation, daily use, configuration, metadata management, schema validation, and the schema lifecycle. Troubleshooting and a command reference appendix close the document.

For security architecture — trust boundaries, redaction, AI-assisted detection — see `docs/SECURITY.md`. That document is intentionally separate; it answers "what guarantees does this give me?", while this manual answers "how do I use it?".

2 Installation

vaultctl ships as a standalone binary, as a Python package, and as source. Pick the path that matches how the project's environment is managed.

2.1 Standalone Binary

The simplest install. One file, no Python runtime needed:

```
curl -fsSL \
  https://github.com/cdds-ab/vaultctl/releases/latest/download/vaultc
↪  tl-linux-amd64
↪  \
  -o /usr/local/bin/vaultctl
chmod +x /usr/local/bin/vaultctl
vaultctl --version
```

The macOS arm64 build is published under the name `vaultctl-macos-arm64` at the same release URL pattern. Releases also include a `checksums.sha256` file for verification.

2.2 From Source

If a project already manages Python tooling with uv:

```
uv tool install \
  --from git+https://github.com/cdds-ab/vaultctl.git vaultctl
```

Or, for development:

```
git clone https://github.com/cdds-ab/vaultctl.git
cd vaultctl
uv sync
uv run vaultctl --help
```

vaultctl requires Python 3.13 or newer when installed from source.

2.3 Required External Tools

vaultctl orchestrates two external command-line tools. Both must be on `PATH`.

`ansible-vault` Performs the actual encryption and decryption. Install via your distribution's package manager or `pip install ansible-core`. Without it, vaultctl exits with a clear error on any operation that touches `vault.yml`.

`cue` ***(optional, for schema features)*** Enables `vaultctl validate` and the `vaultctl schema` subcommands. When `cue` is missing, schema features degrade gracefully — `validate` still runs the cross-file consistency check, and the schema commands report a clear setup instruction. Install from https://cuelang.org/docs/install/.

2.4 Self-Update

Standalone binaries can update themselves:

```
vaultctl self-update
```

Updates verify SHA-256 checksums from the GitHub release, refuse downgrades, and refuse releases without published checksums. Source installs use their respective package manager — `self-update` reports an error and exits.

3 First Steps

The shortest useful tour: create a new vaultctl project, add a secret, and read it back.

3.1 Initialise a New Project

In an empty directory:

```
vaultctl init
```

This creates two files and prompts for a vault password:

```
.vaultctl/
└── config.yml          # vaultctl's own configuration
vault.yml               # encrypted secrets (empty)
vault-keys.yml          # secret metadata (empty)
```

Open `.vaultctl/config.yml`. The defaults assume the vault and keys files live next to it; for an Ansible project, you'll typically point them into `inventory/group_vars/`. See **Configuration** for details.

3.2 Add and Read a Secret

```
vaultctl set api_token "abc123-secret"
vaultctl get api_token
# → abc123-secret
```

For longer values, read from a file:

```
vaultctl set ssh_key --file ~/.ssh/deploy_id_rsa
```

Or interactively (the value never appears in shell history):

```
vaultctl set db_password --prompt
```

`vaultctl list` shows every key with a one-line summary including type tags for structured entries:

```
api_token              abc123-secret...
db_password            ********
ssh_key       [sshKey] -----BEGIN OPENSSH...
```

3.3 Import an Existing Vault

Already have an Ansible Vault? Point `init` at it:

```
vaultctl init --vault-file inventory/group_vars/all/vault.yml
```

vaultctl decrypts the existing file locally, generates the metadata skeleton, and offers to auto-detect entry types. No secrets leave the process during this scan.

4 Configuration

vaultctl's configuration lives in `.vaultctl/config.yml`, a small YAML file at the project root. The directory `.vaultctl/` is reserved for vaultctl's own files (config and CUE schemas); secrets and metadata stay where Ansible expects them.

4.1 Discovery

When you run any `vaultctl` command, the configuration is located by the first match in this order:

1. The `$VAULTCTL_CONFIG` environment variable, if set, points to a config file directly.
2. Walking upward from the current directory to the git root, the first `.vaultctl/config.yml` encountered wins.
3. The user-global fallback at `~/.config/vaultctl/config.yml` is tried last.

Most projects use option 2 — drop a `.vaultctl/` directory at the repo root and forget about it.

4.2 Anatomy of `config.yml`

```yaml
# .vaultctl/config.yml
vault_file: inventory/group_vars/all/vault.yml
keys_file:  inventory/group_vars/all/vault-keys.yml

password:
  env:  VAULT_PASS                  # tried first
  file: ~/.ansible-vault-pass       # then file
  cmd:  pass show project/vault     # then command

# Optional, for AI-assisted type detection.
ai:
  endpoint:    https://api.openai.com/v1/chat/completions
  model:       gpt-4o-mini
  api_key_cmd: pass show openai/api-key
  consent:     true
```

`vault_file` and `keys_file` are paths resolved relative to the project root — the parent of `.vaultctl/` — not relative to `.vaultctl/` itself. This convention keeps secret files at their natural location next to the inventory data.

4.3 Password Resolution Chain

The vault password is resolved by trying each configured source in order. The first non-empty result wins:

1. `password.env` — read from the named environment variable.
2. `password.file` — read from the file at the given path. `~` expands to the home directory.
3. `password.cmd` — execute the command in a shell and use the trimmed standard output.

If all configured sources are empty or unset, vaultctl exits with an error listing exactly which sources were tried. The resolved password lives only in process memory and is never written to disk by vaultctl itself.

5 Daily Use

The commands you'll reach for every day.

5.1 Reading

```
vaultctl list                 # all keys with descriptions
vaultctl list -f api          # filter by regex on names + metadata
vaultctl get db_password      # print a single value
vaultctl describe db_password # show metadata: rotate, consumers,
 ↪  expiry
```

For structured entries (a key whose value is a dict, e.g. a username/password pair):

```
vaultctl get db_creds                    # YAML-formatted full entry
vaultctl get db_creds --field username    # one field, raw
```

5.2 Writing

```
vaultctl set api_token "abc123"
vaultctl set ssh_key --file id_rsa
vaultctl set db_password --prompt
vaultctl set legacy_token --base64 dGVzdA==
```

By default, overwriting an existing key prompts for confirmation and saves the previous value to `<key>_previous`. Two flags control that:

`--force` Skip the confirmation prompt. Useful in scripts.
`--no-backup` Don't keep the previous value as `<key>_previous`. Use when you don't want a snapshot of the old secret hanging around.

`--expires YYYY-MM-DD` writes the expiry into `vault-keys.yml` alongside the secret update.

5.3 Restoring and Editing

`vaultctl restore <key>` swaps a key's current value with its `_previous` backup — useful if a rotation went wrong. `vaultctl edit` opens the decrypted vault in `$EDITOR` via `ansible-vault edit`; on save the file is re-encrypted in place.

5.4 Searching

`vaultctl search <pattern>` runs a regex across both metadata and decrypted values:

```
vaultctl search 'admin'           # matches descriptions, key names,
↪  values
vaultctl search 'admin' --context # show parent objects of matches
```

Search is local; nothing is sent anywhere.

5.5 Deleting

```
vaultctl delete obsolete_key
```

A confirmation prompt asks before removing. With `--force`, the deletion is silent.

6 Metadata and Expiry

Secret values live in `vault.yml`. Their *metadata* — the human-readable context — lives in a sibling file `vault-keys.yml`.

6.1 Anatomy of `vault-keys.yml`

```
vault_keys:
  api_token:
    description: "API token for service X"
    type:        secretText
    rotate:      "365d"
    expires:     "2026-12-01"
    consumers:   ["host01", "host02"]
    rotate_cmd:  "Web UI → Settings → Regenerate"

  db_creds:
    description: "Database credentials, primary cluster"
    type:        usernamePassword
    rotate:      "90d"
    consumers:   ["app01", "app02"]
```

Every field is optional. The minimum useful entry is a one-line `description`. The metadata file is not encrypted — review it like any other config file in the repository.

6.2 Recognised Fields

`description` Free-form text, ideally one sentence about what this secret is for.

`type` One of `usernamePassword`, `sshKey`, `certificate`, `secretText`. Drives display formatting and is enforced by the bundled CUE schema.

`rotate` Recommended rotation period. Any string; common forms are `30d`, `90d`, `365d`, `never`.

`expires` Either a date (`YYYY-MM-DD`) or a full RFC 3339 timestamp. Used by `vaultctl check`.

`consumers` List of hosts or services that read this secret. Helps when planning rotation.

`rotate_cmd` Free-form note about how to rotate the secret. A URL, an internal-tool name, or a shell snippet.

6.3 Checking Expiry

`vaultctl check` reads `vault-keys.yml` and reports any keys that are expired or expiring within a window:

```
vaultctl check                # default: 30 days warning, exit 1 if
↪  any expired
vaultctl check --warn-days 60
vaultctl check --json         # machine-readable output for CI
↪  integration
vaultctl check --quiet        # exit code only; no output
```

The exit code is `1` if any key is expired or within the warning window, `0` otherwise. That makes the command suitable as a cron or CI gate.

6.4 Updating Metadata

`vaultctl set --expires` adjusts the `expires` field along with a value update. For other fields, edit `vault-keys.yml` directly — it's plain YAML.

`vaultctl describe <key>` is the read counterpart:

```
api_token
  description: API token for service X
  type:        secretText
  rotate:      365d
  expires:     2026-12-01 (in 217 days)
  consumers:   host01, host02
  rotate_cmd:  Web UI → Settings → Regenerate
```

7 Schema Validation

Schemas catch typos, wrong types, and structural inconsistencies before they reach production. vaultctl uses CUE — the same language Kubernetes, Tekton, and other tooling chose for similar problems.

When the `cue` binary is on `PATH`, four classes of check are available:

- **Config** — `.vaultctl/config.yml` against the bundled `#Config` schema. Catches typos like `pasword:`.
- **Metadata** — `vault-keys.yml` against the bundled `#KeysFile` schema. Rejects unknown fields, unknown `type` values, malformed `expires` dates.
- **Content** — `vault.yml` against either the bundled `#VaultFile` schema (permissive) or a project-local override.
- **Cross-consistency** — every key in `vault.yml` has metadata in `vault-keys.yml` and vice versa. Pure-Python; runs even without `cue`.

7.1 The Validate Command

```
vaultctl validate                 # all four checks
vaultctl validate --skip-content  # don't decrypt vault; just check
↪  structure
```

Exit codes:

- `0` — all checks passed.
- `1` — at least one violation; specifics are printed to stderr with the source file in brackets.

When `cue` is missing, schema checks are skipped with a clear warning and the cross-consistency check still runs. The exit code reflects the cross-check outcome.

7.2 Bundled vs Custom Schemas

Bundled schemas under the `vaultctl` package cover the universal cases (config and metadata structure). The default content schema is permissive — any string, any structured object — because vault content varies wildly across projects.

Drop your own CUE files into `.vaultctl/` to override the defaults:

```
.vaultctl/
├── config.yml
├── vault.cue               # overrides #VaultFile
├── vault.constraints.cue  # additional rules; CUE merges with vault.cue
└── keys.cue                # overrides #KeysFile
```

The override is automatically picked up by the next `vaultctl validate` run. CUE's package merging means sibling `.cue` files in the same directory are unified at validation time — useful for the **baseline plus constraints** pattern explained in the next chapter.

7.3 Example: Enforce a Minimum Password Length

A `vault.constraints.cue` file in `.vaultctl/`:

```
package vaultctl

#VaultFile: {
    db_password: =~"^.{12,}$"
}
```

Now any vault entry called `db_password` shorter than twelve characters fails validation. Combine with the auto-generated baseline (next chapter) and you get both structure and content rules in one place.

8 Schema Lifecycle

A schema written by hand is hard to keep in sync with a real vault. vaultctl ships a small lifecycle around CUE schemas: derive a baseline from the current vault, detect drift, and react to structural changes during normal `set` operations.

8.1 Bootstrap: `schema infer`

```
vaultctl schema infer
```

Decrypts the vault, walks its content, and writes a closed `#VaultFile` definition to `.vaultctl/vault.cue`. The output covers exactly the keys and shapes present at the moment of the inference. Adding a new key without updating the schema makes `vaultctl validate` fail — that's the value: structural change becomes a deliberate, reviewable step.

The generated file carries a header comment marking it as auto-managed:

```
// AUTO-GENERATED by `vaultctl schema infer` — do not hand-edit.
// Project-specific constraints belong in vault.constraints.cue.
```

`--force` overwrites an existing baseline. `--output PATH` writes elsewhere (useful in CI for ephemeral comparison).

8.2 Drift Detection: `schema sync`

```
vaultctl schema sync           # diff-only; exit 1 on drift
vaultctl schema sync --apply   # rewrite the baseline
```

`schema sync` re-derives the schema from the current vault and compares to the existing `.vaultctl/vault.cue`. The exit codes:

- `0` — baseline matches the vault.
- `1` — drift detected (or fresh schema written with `--apply`).
- `2` — no baseline exists; suggests `vaultctl schema infer` first.

Without `--apply`, the command prints a unified diff. That's the format CI pipelines can flag back to a developer in a code review:

```
--- /home/.../vault.cue (current)
+++ /home/.../vault.cue (inferred)
@@ -3,4 +3,5 @@
 #VaultFile: {
        existing_key: string
+       new_key: int
 }
```

8.3 Schema-Aware `set`

When a baseline exists, `vaultctl set` checks the new vault content before encrypting. If a value introduces a structure the schema doesn't cover, you see the diff and decide:

```
$ vaultctl set new_token "abc123"
Schema does not cover this change:
[unified diff omitted]

Extend .vaultctl/vault.cue with this change? [y/N]:
```

Three flags control this for non-interactive use:

`--extend-schema` Rewrite the baseline silently and continue. Useful in scripted bulk updates where every new key is expected to extend the schema.

`--no-extend-schema` Skip the prompt and proceed with drift. The schema diff is still printed to stderr so it shows up in CI logs. `vaultctl validate` will flag the drift later.

`--force` ***(without either of the above)*** Behaves as `--no-extend-schema`. Safer non-interactive default — the operator opts in to schema changes explicitly.

8.4 Baseline Plus Constraints

The auto-managed baseline (`vault.cue`) and the hand-edited constraints (`vault.constraints.cue`) coexist by design. CUE merges all `.cue` files in the same package at validation time, so:

- `schema infer` and `schema sync --apply` rewrite *only* `vault.cue`.
- Hand-edits in `vault.constraints.cue` survive baseline regeneration unchanged.
- The effective schema at validation time is the unification of both files.

Use this split to keep regex constraints, value ranges, and required-field rules separate from the structural baseline.

9 Type Detection

Structured vault entries fall into a small set of recognisable shapes:

`usernamePassword` A dict with `username` and `password` (and possibly `host`, `port`, etc.).
`sshKey` A string starting with an OpenSSH header, or a dict containing one.
`certificate` A PEM-encoded certificate or chain.
`secretText` The catch-all for free-form scalar secrets — API tokens, passphrases, opaque strings.

`vaultctl detect-types` infers a likely type for each entry by inspecting key names, dict field names, and the first few bytes of values. Results are presented as suggestions, not changes:

```
vaultctl detect-types                    # dry-run summary
vaultctl detect-types --apply            # write detected types into
↪  vault-keys.yml
vaultctl detect-types --confidence high  # show only high-confidence
↪  matches
vaultctl detect-types --json             # machine-readable output
```

9.1 Local vs AI-Assisted

By default, detection runs a local heuristic. For nuanced cases — entries the heuristic can't classify — there's an opt-in AI mode:

```
vaultctl detect-types --ai --yes
```

This sends a **redacted** view of the vault to a configured AI endpoint. Three independent layers prevent secret leakage:

1. **Redaction** replaces every leaf value with `***REDACTED***` before any payload is built.
2. **Explicit field extraction** sends only key names, field names, explicit `type` markers, and Phase 1 heuristic results — never the redacted dict as-is.
3. **Consent dialog** lists exactly what will be sent, the target endpoint, the model, and a SHA-256 hash of the payload.

`vaultctl detect-types --show-payload` decrypts, redacts, builds the payload, prints it (with hash), and exits without sending.

For the full security architecture — trust boundaries, transport enforcement, what is never sent — see `docs/SECURITY.md`. AI-assisted detection is documented there at depth.

10 Troubleshooting

10.1 "No `.vaultctl/config.yml` found"

vaultctl walks upward from the current directory looking for a config file. If you're outside the project, it won't find one.

Fix: `cd` into a directory inside the project, run `vaultctl init` to create one, or set `VAULTCTL_CONFIG=/absolute/path/to/config.yml`.

10.2 "Decryption failed (no vault secrets were found that could decrypt)"

The vault password didn't resolve to anything that `ansible-vault` accepts. Check `.vaultctl/config.yml`:

```
password:
  env:  VAULT_PASS               # this env var must be set
  file: ~/.ansible-vault-pass    # this file must exist and be readable
  cmd:  pass show project/vault  # this command must succeed
```

At least one source must produce the password. The chain is tried top to bottom — first non-empty result wins. The error message lists exactly which sources were tried.

10.3 "cue binary not found on PATH"

Schema features (`validate`, `schema infer`, `schema sync`, schema-aware `set`) all need `cue`. Install from https://cuelang.org/docs/install/.

When `cue` is missing, `vaultctl validate` still runs the cross-file consistency check. The other schema commands print a setup hint and exit.

10.4 `vaultctl init` Overwrote My Password Config

`init` writes a fresh `.vaultctl/config.yml` with default password sources. If you re-run it on an existing project, the existing config is replaced. Pass `--force` to confirm the intent.

To avoid the overwrite, edit `.vaultctl/config.yml` directly instead of re-running `init`.

10.5 `self-update` Says "Only Available for Standalone Binaries"

You installed via `pip` or `uv tool install`, not the GitHub Releases binary. Update through the package manager:

```
uv tool install --force \
  --from git+https://github.com/cdds-ab/vaultctl.git vaultctl
```

11 Command Reference

A condensed alphabetical table. Run `vaultctl <command> --help` for full option lists.

Command	Purpose
`vaultctl init`	New project or import an existing vault
`vaultctl list [-f REGEX]`	List keys with descriptions; optional filter
`vaultctl get KEY [--field F]`	Print a key's value (full or single field)
`vaultctl set KEY [VALUE]`	Add/update a key (`--prompt`, `--file`, `--base64`)
`vaultctl delete KEY`	Remove a key (`--force` skips confirmation)
`vaultctl describe KEY`	Show metadata for a key
`vaultctl restore KEY`	Swap current value with `_previous` backup
`vaultctl edit`	Open vault in `$EDITOR`
`vaultctl check`	Report expired or expiring keys
`vaultctl search PATTERN`	Regex search across keys, values, metadata
`vaultctl detect-types`	Auto-detect entry types (`--apply`, `--ai`)
`vaultctl validate`	Run all schema and cross-file checks
`vaultctl schema infer`	Generate a CUE schema baseline
`vaultctl schema sync`	Detect drift; `--apply` to update baseline
`vaultctl self-update`	Update standalone binary to latest release
`vaultctl completion SHELL`	Print shell completion script

11.1 Universal Flags

`--config PATH` Override config file discovery.
`--vault-file PATH` Override the vault path from config (one-shot override).
`--version` Print version and exit.
`--help` Print help for any command or subcommand.

11.2 Configuration Reference

A complete `.vaultctl/config.yml`:

```
vault_file: inventory/group_vars/all/vault.yml
keys_file:  inventory/group_vars/all/vault-keys.yml

password:
  env:  VAULT_PASS
  file: ~/.ansible-vault-pass
  cmd:  pass show project/vault

ai:
  endpoint:    https://api.openai.com/v1/chat/completions
  model:       gpt-4o-mini
  api_key_cmd: pass show openai/api-key
  consent:     true
```

All fields are optional except — pragmatically — at least one entry under `password`.

11.3 Where to Go Next

For security architecture and trust boundaries, read `docs/SECURITY.md`. The `cdds-ab/vaultctl` repository on GitHub is where issues, releases, and source live. Contributions are welcome.

This manual was generated with `pandoc --pdf-engine=xelatex` from `docs/manual.md`. Rebuild with `bash docs/build_manual.sh` after editing.